

# Controlled experimentation on adaptations of pair programming

Madeline Ann Domino · Rosann Webb Collins ·  
Alan R. Hevner

Published online: 21 February 2007  
© Springer Science+Business Media, LLC 2007

**Abstract** The use of agile methods is growing in industrial practice due to the documented benefits of increased software quality, shared programmer expertise, and user satisfaction. These methods include pair programming (two programmers working side-by-side producing the code) and test-driven approaches (test cases written first to prepare for coding). In practice, software development organizations adapt agile methods to their environment. The purpose of this research is to understand better the impacts of adapting these methods. We perform a set of controlled experiments to investigate how adaptations, or variations, to the pair programming method impact programming performance and user satisfaction. We find that method variations do influence programming results. In particular, better performance and satisfaction outcomes are achieved when the pair programming is performed in face-to-face versus virtual settings, in combination with the test-driven approach, and with more experienced programmers. We also find that limiting the extent of collaboration can be effective, especially when programmers are more experienced. These experimental results provide a rigorous foundation for deciding how to adapt pair programming methods into specific project contexts.

**Keywords** Pair programming · Test-Cases First · Controlled experiments · Agile methods

Agile software development processes are becoming increasingly viable alternatives to more disciplined, plan-driven development processes. The interest is driven by the promise of higher quality code produced in shorter periods of time, with high levels of developer satisfaction. A great deal of uncertainty remains however, on how best to structure an effective, context-specific process that remains faithful to the principles of agility [2, 5]. Few research studies report empirical results that can be used to guide project managers in these critical decisions [9, 11].

A key premise of agility in software development is the freedom to adapt processes and methods to the needs of a specific project. Previous research has referred to such adaptation as ‘method engineering’ [26, 27] or ‘method tailoring’ [10]. Turk, France and Rumpe [30, p. 73] identify one of the central underlying assumptions of agility as “developers have the experience needed to define and adapt their processes appropriately.” Thus, the need for research on how to effectively adapt agile methods is clear.

In this research we study the adaptation of pair programming concepts in agile development projects. We conducted a series of studies that investigate pair programming variations that are currently being employed in practice: in the work setting, preparation for coding, and extent of collaboration. Section 1 describes the pair programming method and the potential variations of the method. Existing research on pair programming is surveyed. Our research study is presented in Section 2. We perform a set of controlled

---

M. A. Domino  
Management Information Systems, College of Business  
Administration, University of Central Florida, Orlando,  
FL 32816, USA

R. W. Collins · A. R. Hevner (✉)  
Information Systems and Decision Sciences, College of  
Business Administration, University of South Florida,  
Tampa, FL 33620, USA  
e-mail: ahevner@coba.usf.edu

experiments to investigate how adaptations, or variations, to the pair programming method impact programming performance and user satisfaction. The experimental results are presented and discussed in Section 3. The implications of this research to practice are discussed in Section 4 along with research limitations and future directions.

## 1 Agile programming methods

Software development processes and methods support how an organization chooses to organize people and resources to create and maintain software applications. Developer collaboration, which is inherent to some level in most methods, has long been acknowledged by practitioners as a good development approach [7]. Agile programming methods, in particular, require intense levels of personal interaction among software developers during the design, coding, and testing of software programs. Extreme programming, one of the most widely used agile processes, entails the use of pair programming as well as test-driven development [16]. The use of agile methods is not without risks and limitations and some recommend careful consideration and incorporation of agile and more plan-driven methods in projects [4, 29]. Therefore, it is especially important that research reveal how the different aspects of agile methods, used in varied combinations, impact development outcomes.

### 1.1 Pair programming

In this research, we employ pair programming [32] and investigate adaptations, or variations, in its implementation. In pair programming, the collaborative team is limited to two programmers who work together the entire time, using a prescribed protocol of clearly defined roles to produce a software artifact. One developer takes the role of the driver, writing the code and using the keyboard, while the other functions as the navigator, monitoring results, looking for specific details and strategic defects. Periodically partners switch roles, resulting in a highly interactive, adaptive development process [3].

Pair programming is gaining organizational interest as the software industry continues to strive for better production quality and productivity. Ford Motor Company, Caterpillar Corporation, and other large software development groups [6] have reported using pair programming or some closely related variation of the method. However, skeptics of pair programming suggest that utilizing two developers the entire time for one

programming task may be too resource intensive. This point has influenced some organizations to reconsider how they implement pair programming. For example, in some industrial settings, pair programming is implemented by having a set of developers brainstorm together and then write code individually [17]. Proponents of this variation of pair programming claim the approach produces the same high quality code without being as resource intensive. Thus, some variations of the pair programming method may be viewed as more cost effective, while preserving the other benefits of improved software quality and high developer satisfaction.

### 1.2 Research on pair programming

Research studies on pair programming provide some evidence of its benefits and costs. Flor and Hutchins [12] theorize that distributed cognition explains enhanced performance outputs. Distributed cognition refers to the knowledge representation both inside the heads of the individuals and in the world, and the propagation of knowledge between individuals and artifacts. In an intensive process study of two developers working on a maintenance task, the researchers found evidence of distributed cognition in pair programming: intensive communication in order to share goals, share memories, and expand the problem solving space. These programmers had to communicate in order to collectively formulate the maintenance problem to solve; share their different, prior development experiences in relevant tasks; and consider alternate solutions to the problem. According to distributed cognition, these communications create a richer problem-solving environment that explains higher levels of performance with pair programming.

Other experimental studies also find higher performance from pair programming. In a laboratory experiment, Nosek [22] compared performance outcomes for five experienced programmers working alone on a single, 45-minutes task programming task to those of five pairs of experienced programmers. According to his findings, the collaborative pairs produced higher quality code and completed the task forty percent more quickly. Williams et al. [32] conducted an experiment in which computer science students utilized pair programming to complete three experimental tasks. They found that after an initial “jelling period”, paired programmers produced higher quality code (15% less defects), in shorter time (completed in half the time), with a reported rate of 95% higher levels of developer satisfaction. These study results are limited, however, by the lack of control in the experiment: subjects did not perform the experimental tasks in a laboratory setting

(and so there is no assurance that the pairs followed the driver-navigator roles and worked together on all tasks), and time on task was self reported.

The results of another experimental evaluation of pair programming calls into question these positive findings. Nawrocki and Wojciechowski [21] found pair programming to be less efficient than claims made by earlier researchers with no real difference in quality outcomes. Additionally, little empirical evidence has been offered to explain the exact sources of the gains in quality that have been reported. Flor and Hutchins' [12] thesis on distributed cognition offers some glimpse as to why performance outcomes are improved. However, given these inconclusive findings on pair programming and the limitations in the research designs employed, the call for more research on the topic is warranted.

In summary, most studies show that pair programmers produce better code and enjoy higher levels of satisfaction at the cost of some additional staffing overhead [1, 14, 22, 31, 32]. However, other studies show either minimal or overall negative impacts of introducing agile methods, such as pair programming and test-driven development [20]. One reason for the mixed results may be the variations in the method tested, especially if some essential feature of the method is not employed. Two developers working concurrently on programming tasks may, on the surface, appear to be pair programming. But Parrish et al. [23] find that programming pairs are not naturally productive, and increased collaboration in development work without a clear method for that collaboration, is associated with significantly lower productivity. This result demonstrates that methods have essential features: specific aspects of the method that must be employed in order to achieve positive returns with its implementation.

The goal of our research is to understand what aspects of the pair programming method produce positive outcomes in order to assure that the chosen variations in the method-in-use are faithful to those elements. We ask: How do the variations of pair programming contribute to effective software development? Success in this context includes both quality of code produced by the pair programming dyad and their satisfaction with using the pair programming method.

## 2 Design of the controlled experiments

The series of controlled experiments reported in this paper test whether variations in pair programming impact the performance and satisfaction outcomes from pair programming. The variations used in these experiments are described in the following section.

### 2.1 Pair programming variations

Variations, or adaptations, of the standard pair programming method can occur by design, but they also may result from naturally occurring differences between developers. Organizations design software development projects, and part of that design may be the adaptation of development methods to fit work setting and resources. Naturally occurring variations include individual differences between the developers employing a method and in those developers' faithfulness to the method. In this study organizational adaptations are used as experimental manipulations, and the naturally occurring variations are measured.

The standard pair programming method dictates co-located, or face-to-face, collaborative communication, as when two developers work together at the same workstation. Today's dynamic and global business environments do not always support face-to-face, same location collaboration. Hence we include virtual pair programming as a variation of the work setting.

A number of advocates of agile methods (in particular, extreme programming) advocate a test-driven approach to programming, such as the use of test cases, or structured logic, to precede the writing of the associated code [16]. This test-driven approach is believed to be associated with lower defect rates [18]. Thus we include writing test cases before producing code as a variation in our experiments. It is unknown if this test-cases-first prescription is essential because it is a structured problem solving approach, or if the same result can be achieved with a less-structured activity like brainstorming together prior to writing code [17]. In order to investigate this issue, study dyads were varied on whether they create test cases first or brainstorm together first instead of creating test cases.

The final manipulation investigates whether or not developers should collaborate fully (creating test cases and coding as a pair), or partially (creating test cases together and then working alone on coding), or not at all (creation of test cases and coding alone). This is of particular interest for managers who view full collaboration of developers in pair programming as a strain on limited development resources, since they appear to duplicate work effort [23].

To date, little empirical evidence is available on the impacts of individual differences and variations in the use of the pair programming method on performance. In this study, we measure each individual's information technology experience, since prior research suggests that, in general, experience has a strong positive link to performance [15]. Should experience be shown to be a significant factor in pair programming success, then the

relative experience of developers can be used as a selection criterion for pair programming teams.

Research shows that when individuals and groups use a work structure like the pair programming method, they *appropriate* (adapt) it. Faithful appropriations are those changes that retain the original spirit or intent of the method, while ironic adaptations violate the essential features of the structure [13, 24, 25]. This naturally-occurring variation is addressed in two ways. First, all studies were conducted in the laboratory, with a low researcher-to-subject ratio to enable monitoring of subjects to be sure that both individuals in the dyads were working and that only the subjects in the driver-role were using the keyboard. Second, in those groups using the test-driven approach, we measure the extent of test case creation as an indicator of faithfulness to this method.

## 2.2 Research method and experimental design

A series of controlled experiments were designed and conducted at a large urban university located in the southeastern United States. Two hundred and sixteen subjects (216) were recruited for the study. Table 1 summarizes the study design. Participants were drawn from several different groups in order to include various IT experience levels in the study. The initial group (Group 1) consisted of developers who were recruited for the study because they have several years of IT experience. Five additional groups (Groups 2–6) were full- and part-time graduate and undergraduate students majoring in management information systems (MIS) and accounting information systems (AIS) who volunteered to participate in the study. The student subjects in these groups have varying levels of IT

experience. As an incentive to participate in the study, student subjects who completed the experiment received credit towards their final course grade. Subjects who chose not to participate were allowed to complete an alternative assignment.

For logistical reasons, groups were not run at the same time, but within a two-month time frame, in the sequence shown in Table 1. For example, the virtual group 3 required two separate computer laboratories with appropriate collaboration software (Groove) and equipment (headsets with microphones, no keyboards or mice for navigator role subjects). It was also important to run the study groups at separate times so that researchers could monitor subjects to assure participation by both members of the programming pair.

Given the logistical constraints, the research was conceptualized in two phases (as opposed to a single, factorial design study). The first phase (Groups 1–3) focused on the impact of work setting, with experienced software developers recruited for group 1 to ensure variation in experience. The second phase focused on the impact of preparation for coding and added to the full-collaboration groups of phase one to ensure variation in levels of collaboration. The goal is to make a series of contrasts for the variables of interest, while holding the other variables constant. For example, Group 3 (virtual work setting) is not included in the analysis of extent of collaboration—only face-to-face groups are used in the contrast.

Subjects in Groups 1–4 and 6 were randomly assigned to a designated programming pair (dyad) and to initial roles of driver and navigator. Subjects working in pairs switched roles for the second task. Group 5 subjects worked alone. All participants were assigned two experimental tasks (labeled Task I and Task II).

**Table 1** Experimental design

| Group                 | Study participants                                       | Variations (experimental manipulations)                                         |                          |                |
|-----------------------|----------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------|----------------|
|                       |                                                          | Preparation for coding                                                          | Extent of collaboration  | Work setting   |
| 1                     | Experienced software developers                          | Test-driven approach                                                            | Full collaboration       | Face-to-face   |
| 2                     | Undergraduate MIS students                               | Test-driven approach                                                            | Full collaboration       | Face-to-face   |
| 3                     | Undergraduate MIS students                               | Test-driven approach                                                            | Full collaboration       | Virtual        |
| 4                     | Mixed graduate and undergraduate students in MIS and AIS | Test-driven approach, as a pair; code alone                                     | Partial collaboration    | Face-to-face   |
| 5                     | Mixed graduate and undergraduate students in MIS and AIS | Test-driven approach, alone; code alone                                         | No collaboration         | Not applicable |
| 6                     | Mixed graduate and undergraduate students in MIS and AIS | Brainstorming first, as a pair; code alone                                      | Partial collaboration    | Face-to-face   |
| Experimental sequence | Training & initial questionnaire                         | Task I & Task II                                                                | Final questionnaire      |                |
| Measures              | Years of IT experience                                   | Performance: accuracy of code<br>Faithfulness to method: accuracy of test cases | Satisfaction with method |                |

Two tasks were included in the experiment in order to allow for team jelling and to vary the difficulty of the tasks (Task II is designed to be more difficult, and subjects ranked Task II as more difficult in the manipulation check). The order of the experimental tasks was randomly assigned for all subjects.

All subjects received training appropriate to the experimental setting (e.g., how to use the collaboration software, how to create test cases, how to brainstorm, the navigator and driver roles in pair programming) and followed the designated protocol for their treatment group. Subjects were asked to write pseudocode in order to deal with unknown differences within pairs in their knowledge and experience with specific programming languages. As a result, the experimental tasks are classified as “Create—Design” tasks following the Sjöberg et al. [28] taxonomy. During the experimental tasks, only the drivers in each pair had access to the keyboard. Forty-five minutes were allotted for the completion of each programming task.

Group 1 subjects were selected for professional IT experience; they worked in pairs, face-to-face, using the test-driven approach. Groups 2–6 used undergraduate and graduate classes as subjects, and so in these groups the IT experience level was expected to vary naturally. In addition to this naturally-occurring variation, three variations of pair programming were employed as experimental manipulations: (1) use of a test-driven approach or brainstorming as preparation to code, (2) extent of collaboration, and (3) work setting, whether face-to-face or virtual.

### 2.3 Data collection and analysis

A pilot study was conducted prior to beginning the experiment in which scripts, questionnaires, and experimental tasks were tested. The experimental tasks have been used in prior research and many of the items in the questionnaires were adapted from existing instruments. (See Appendix I for the experimental tasks.) After a review of the results, changes were incorporated into the experimental materials as appropriate.

We evaluated two outcomes: performance on task and individual satisfaction with the method. Performance on task was measured by the accuracy of the code produced on each programming task. A scoring template was developed by the researchers to rate the two programming outcomes on a scale of 1–10 (see Appendix II). In principle, the more complete and accurate the code was, the higher the score was. Two independent raters were trained and used to evaluate task performance. There was a high level of inter-rater

reliability on all tasks. Inter-rater reliability varied by pair (95–100%) and is based on the percentage of agreement for each item rated.

In order to measure satisfaction, subjects rated their level of satisfaction with their assigned method on 7 point Likert scale (1 = not satisfied and 7 = very satisfied), adapted from Venkatesh and Vitalari [31] and McGrath [19]. Years of IT job experience was measured on an initial questionnaire. Faithfulness to the method was measured by the number of correct test cases created by pairs in the experimental groups that used the test-first approach (Groups 1–5).

## 3 Results and discussion

The descriptive data for all groups are presented in Table 2. Tests for significant differences between groups are shown in Tables 3 through 8. The Kruskal–Wallis non-parametric statistic is used because the performance dependent variable is not normally distributed. The Kruskal–Wallis mean rank statistic reflects each group’s average rank when raw scores for all groups are combined into one rank list. The  $\chi^2$  test statistic evaluates whether or not there is a significant difference between the mean rank performances between the groups. To aid understanding, the differences in performance and satisfaction between groups are shown in Tables 3–8 via the raw mean performance score (scale = 1–10) or the raw mean satisfaction score (scale = 1–7), for each grouping.

The overall test for differences between groups on performance is significant (Task I,  $\chi^2 = 32.91$ ,  $p = 0.000$ ; Task II,  $\chi^2 = 28.68$ ,  $p = 0.000$ ), which supports the subsequent analysis reported in this section. These analyses investigate the impacts on group performance and developer satisfaction of the work setting, preparation for coding, extent of collaboration, developer experience, and developers’ faithfulness to method.

### 3.1 Adaptation in work setting: virtual versus face-to-face

In order to compare the impact of work setting on task performance, we analyze the performance results for all subjects using pair programming method combined with the test-driven approach and full collaboration (Groups 1–3). For this analysis, we compare the performance outcome of Groups 1 and 2, whose pairs worked in a co-located, or face-to-face work setting, to the performance outcomes of the

**Table 2** Descriptive data

| Group | N*            | Experience                           | Individual satisfaction with method for Tasks I & II | Task I Performance: accuracy of code | Task II Performance: accuracy of code |
|-------|---------------|--------------------------------------|------------------------------------------------------|--------------------------------------|---------------------------------------|
| 1     | 20 (10 pairs) | Mean = 4.55 years<br>St. Dev. = 2.87 | Mean = 6.20<br>St. Dev. = 0.77                       | Mean = 5.30<br>St. Dev. = 3.27       | Mean = 4.55<br>St. Dev. = 2.99        |
| 2     | 36 (18 pairs) | Mean = 1.39 years<br>St. Dev. = 2.10 | Mean = 5.31<br>St. Dev. = 1.04                       | Mean = 3.81<br>St. Dev. = 2.18       | Mean = 2.39<br>St. Dev. = 2.41        |
| 3     | 40 (20 pairs) | Mean = 2.72 years<br>St. Dev. = 2.76 | Mean = 4.71<br>St. Dev. = 1.10                       | Mean = 2.03<br>St. Dev. = 1.63       | Mean = 1.60<br>St. Dev. = 1.66        |
| 4     | 42 (21 pairs) | Mean = 3.71 years<br>St. Dev. = 1.31 | Mean = 4.92<br>St. Dev. = 1.00                       | Mean = 4.95<br>St. Dev. = 2.60       | Mean = 3.64<br>St. Dev. = 1.89        |
| 5     | 32            | Mean = 4.22 years<br>St. Dev. = 1.45 | Mean = 4.31<br>St. Dev. = 0.99                       | Mean = 4.84<br>St. Dev. = 2.01       | Mean = 4.06<br>St. Dev. = 2.48        |
| 6     | 46 (23 pairs) | Mean = 3.91 years<br>St. Dev. = 3.91 | Mean = 4.97<br>St. Dev. = 1.12                       | Mean = 4.33<br>St. Dev. = 2.09       | Mean = 4.13<br>St. Dev. = 1.92        |

\* Experience and Satisfaction were measured at the individual level. Performance was measured based on the accuracy of code produced by the pair (Groups 1–4 & 6) or individual (Group 5)

**Table 3** Comparison of task performance in face-to-face and virtual work settings

| Variation in work setting                                                                              | N (Pairs)     | Performance Task I mean rank (raw mean) | Performance Task II mean rank (raw mean) |
|--------------------------------------------------------------------------------------------------------|---------------|-----------------------------------------|------------------------------------------|
| Pair programming, test-driven, full collaboration, & face-to-face work setting (combined Groups 1 & 2) | 28 pairs      | 29.00 (4.31)                            | 27.50 (3.16)                             |
| Pair programming, test-driven, full collaboration, & virtual work setting (Group 3)                    | 20 pairs      | 18.20 (2.03)                            | 20.30 (1.60)                             |
| Kruskal–Wallis test                                                                                    | $\chi^2$<br>p | 7.11<br>0.008                           | 3.17<br>0.075                            |

**Table 4** Comparison of task performance for variations in preparations before coding

| Variation in preparation to code                                   | N (Pairs)     | Performance on Task I mean rank (raw mean) | Performance on Task II mean rank (raw mean) |
|--------------------------------------------------------------------|---------------|--------------------------------------------|---------------------------------------------|
| Structured: create test cases together, write code alone (Group 4) | 21 pairs      | 81.43 (4.95)                               | 72.85 (3.64)                                |
| Unstructured: brainstorm together, write code alone (Group 6)      | 23 pairs      | 72.43 (4.33)                               | 82.24 (4.13)                                |
| Kruskal–Wallis test                                                | $\chi^2$<br>p | 1.016<br>0.314                             | 1.373<br>0.241                              |

**Table 5** Comparison of task performance for variations in extent of collaboration

| Variation in extent of collaboration                                    | N              | Performance on Task I mean rank (raw mean) | Performance on Task II mean rank (raw mean) |
|-------------------------------------------------------------------------|----------------|--------------------------------------------|---------------------------------------------|
| Full collaboration: create test cases and code together (Groups 1 & 2)  | 28 pairs       | 39.38 (4.31)                               | 42.93 (3.16)                                |
| Partial collaboration: create test cases together, code alone (Group 4) | 21 pairs       | 55.58 (4.95)                               | 53.14 (3.64)                                |
| No collaboration: create test cases and code alone (Group 5)            | 32 individuals | 56.75 (4.84)                               | 56.84 (4.06)                                |
| Kruskal–Wallis test                                                     | $\chi^2$<br>p  | 6.631<br>0.036                             | 3.583<br>0.167                              |

**Table 6** Impact of developer experience on task performance

| Variation in experience                                                              | <i>N</i> | Performance Task I<br>mean rank (raw mean) | Performance Task II<br>mean rank (raw mean) |
|--------------------------------------------------------------------------------------|----------|--------------------------------------------|---------------------------------------------|
| Pair programming, test-driven approach,<br>face-to-face experience mean = 4.55 years | 10 pairs | 18.40 (5.30)                               | 18.70 (4.55)                                |
| Pair programming, test-driven approach,<br>face-to-face experience mean = 1.39 years | 18 pairs | 12.33 (3.81)                               | 12.17 (2.39)                                |
| Kruskal–Wallis test                                                                  | $\chi^2$ | 3.64                                       | 4.13                                        |
|                                                                                      | <i>p</i> | 0.056                                      | 0.042                                       |

**Table 7** Impact of faithfulness to method on task performance

| Variation in faithfulness<br>to the method            | <i>N</i> | Performance on Task I<br>Mean Rank (Raw Mean) |
|-------------------------------------------------------|----------|-----------------------------------------------|
| Low faithfulness<br>mean = 4.84,<br>Std. Dev. = 3.18  | 19       | 12.03 (1.38)                                  |
| High faithfulness<br>mean = 9.11,<br>Std. Dev. = 0.33 | 9        | 19.72 (5.83)                                  |
| Kruskal–Wallis Test                                   | $\chi^2$ | 5.57                                          |
|                                                       | <i>p</i> | 0.018                                         |

developers using the same method working in a virtual work setting (Group 3).

As seen in Table 3, at an alpha level of 0.10 ( $p < 0.10$ ), the Kruskal–Wallis Test indicates that there is a significant difference between the mean rank performance between the groups on Task I and Task II. The mean rank task performance for the subjects in a face-to-face work setting is significantly higher than the mean rank task performance for subjects working in a virtual setting. Further statistical analysis shows that there is no significant difference in experience between the groups (face-to-face mean = 2.52, std. dev. = 2.83; virtual mean = 2.72; std. dev. = 2.76). Given these results it appears that virtual pair programming, while feasible, requires more research on how to improve its effectiveness.

### 3.2 Adaptation in method: preparation for coding

We evaluated the impact on task performance of adaptation to the pair programming method based on how developers prepare for coding. In the test-driven approach method, a structured preparation approach to write program code is used: the creation of test cases. In at least one organization this has been modified to an unstructured preparation: the developer pair brainstorms about the task and then each developer writes code alone. In order to make a fair comparison on this variation, we compared the performance of Groups 4 and 6. Both groups performed in a face-to-face setting, working together in a pair to prepare for coding and then alone to write code.

At an alpha level of 0.10 ( $p < 0.10$ ), the Kruskal–Wallis Test indicates that there is no significant difference between Groups 4 and 6 on the mean rank performance on Task I and Task II (see Table 4). This result suggests that when the pair works together prior to coding, a structured approach to that preparation (creating test cases) offers no significant advantage over use of a less structured approach (brainstorming).

### 3.3 Adaptation in method: extent of collaboration

This experimental manipulation tested whether task performance varied with the extent of collaboration.

**Table 8** Comparison of developer satisfaction with method for all variations

| Group/method or work setting variation                                      | <i>N</i> | Satisfaction on Task I<br>mean rank (raw mean) | Satisfaction on Task II<br>mean rank (raw mean) |
|-----------------------------------------------------------------------------|----------|------------------------------------------------|-------------------------------------------------|
| 1 Test-driven approach, full collaboration, face-to-face                    | 20       | 164.90 (6.20)                                  | 179.90 (6.20)                                   |
| 2 Test-driven approach, full collaboration, face-to-face                    | 36       | 134.25 (5.61)                                  | 113.47 (5.09)                                   |
| 3 Test-driven approach, full collaboration, virtual                         | 40       | 87.69 (4.63)                                   | 96.38 (4.77)                                    |
| 4 Test-driven approach, partial collaboration, face-to-face                 | 42       | 109.55 (5.11)                                  | 103.58 (4.72)                                   |
| 5 Test-driven approach, no collaboration, face-to-face                      | 32       | 66.73 (4.13)                                   | 84.94 (4.50)                                    |
| 6 Brainstorming preparation to code,<br>partial collaboration, face-to-face | 46       | 107.23 (4.97)                                  | 108.90 (4.96)                                   |
| Kruskal–Wallis test                                                         | $\chi^2$ | 41.63                                          | 26.58                                           |
|                                                                             | <i>p</i> | 0.000                                          | 0.000                                           |

For this analysis we compare developers who collaborated fully (test-driven approach and coding as a pair—Groups 1 and 2), partially (test-driven approach together and then working alone on coding—Group 4), or not at all (test-driven approach and coding alone—Group 5).

As shown in Table 5, at an alpha level of 0.10, the Kruskal–Wallis Test indicates that there is a significant difference ( $p = 0.036$ ) between the mean rank performance between groups on Task I (the easier task), but not for the harder Task II ( $p = 0.167$ ). For both tasks the mean performance of the pairs who collaborated fully was lower than the performance of developers who worked alone or collaborated only in preparation to code. However, if we test just Group 1 pairs, who collaborated fully but also had relatively higher IT experience, against Groups 4 and 5 (partial and no collaboration treatments), there is no statistical significance for either task (Task I,  $\chi^2 = 0.007$ ,  $p = 0.996$ ; Task II,  $\chi^2 = 0.569$ ,  $p = 0.752$ ). Both results suggest that varying the amount of collaboration when developers are more experienced can be done without sacrificing performance.

### 3.4 Developer experience

As indicated in the previous analysis of the impact of extent of collaboration, paired programmers with more experience could vary on extent of collaboration without a performance change. We also evaluate the impact of experience on task performance by comparing outcomes for subjects that utilized the pair programming method and test-driven approach in the face-to-face setting (Groups 1 and 2). There is a significant difference in the mean experience level of the subjects in each group (Group 1 Mean = 4.55, Std. Deviation = 2.87; Group 2 Mean = 1.39; Std. Dev. = 2.10). As shown in Table 6, at an alpha level of 0.10 ( $p < 0.10$ ), the Kruskal–Wallis Test indicates that there is a significant difference between groups for Task I and Task II, with subjects in the more experienced group achieving significantly higher performance outcomes.

### 3.5 Faithfulness to method

In order to compare the impact of developer faithfulness to method on task performance, we analyze the performance results for those developers utilizing the test-driven approach combined with pair programming in a face-to-face setting (Groups 1 and 2). As described earlier, we monitored subjects during the experiment to assure that both pairs participated and were faithful

to their assigned driver or navigator roles. For each developer task we evaluate faithfulness to the test-driven method based upon the number of correct test cases completed by the pair for each programming task. We rank the pairs into two groups (high and low) depending on the number of correct test cases prepared for each programming task. Dyads that prepared greater than 8 correct test cases (out of the 10 asked for in the experimental protocol) were deemed highly faithful. The higher the number of correct test cases prepared by the dyad, the more faithful they were to the method. Nine (9) of the twenty-eight pairs ranked in the top of performance on test cases, for Task 1. Only two (2) of the 28 pairs ranked in the top performance for Task II and therefore this analysis has not been shown.

As seen in Table 7, at an alpha level of 0.10 ( $p < 0.10$ ), the Kruskal–Wallis Test indicates that there is a significant difference between the mean rank performance on Task I: subjects who were highly faithful to the protocol achieved higher performance for Task I. These findings suggest that faithfulness to method may be important to implementing the test-driven approach when combined with pair programming, given concerns regarding productivity loss and constrained resources. This finding suggests that training programmers on the need to be faithful to this method, (i.e., preparing test cases first) is important when implementing the test-driven approach in an organizational setting.

### 3.6 Developer satisfaction

In prior research developers using pair programming reported high levels of satisfaction with the method. Our results confirm prior findings that indicate higher levels of individual developer satisfaction when pair programming is implemented in face-to-face work setting. At an alpha level of 0.10 ( $p < 0.10$ ), the Kruskal–Wallis Test indicates that there is a significant difference between groups on the mean rank satisfaction scores on Task I and Task II (see Table 8). Subjects who used pair programming in a face-to-face setting report the highest levels of satisfaction.

It is interesting to note that the more highly experienced programmers using pair programming in a face-to-face work setting reported higher levels of individual satisfaction than less experienced developers. Our findings indicate that programmers who used variations to pair programming which required partial collaboration (Groups 4 and 6) also report higher levels satisfaction than developers who worked alone the entire time (Group 5).

Programmers who used pair programming in a virtual setting (Group 3) reported the lowest levels of satisfaction to the method. It should be noted that there is no statistical difference in the mean ranks for satisfaction between developers working alone (Group 5) and developers using pair programming in a virtual setting (Group 3). Our findings give strong support to the notion that developers enjoy collaborative work arrangements. It also appears that the challenges of virtual development appear to overshadow the positive impacts of full collaboration.

#### 4 Discussion and future research directions

The results of our research shed considerable insight into the elements that are essential for successful pair programming outcomes. Our findings illustrate that pair programming outcomes are affected by both individual developer differences and the method adaptations used during development.

The results suggest that pair programming works best when working face-to-face instead of virtually. However, variations used in practice to the pair programming method, such as varying the extent of collaboration and how the pair prepares for writing code, may be employed without a negative impact on performance. Consistent with prior research, collaboration is related to higher levels of developer satisfaction, especially when the collaboration is face-to-face between more experienced developers. The research also provides some evidence that faithfully implementing the test-driven approach-pair programming combination has a positive impact on performance.

This study is of value to practitioners in many ways. The results suggest that if collaboration in the form of pair programming is to be used effectively then organizations must consider how best to adapt pair programming to the development environment, particularly where productivity and resource constraints are an issue. For example, partial collaboration with pair programming, in which the pair works together face-to-face to prepare for coding, but then codes alone, was shown to be a viable adaptation of pure pair programming. This adaptation was found to have similar performance outcomes to pure pair programming, but also retained the higher level of satisfaction from working together. However, the virtual setting variation was discovered to have significantly lower performance and lower developer satisfaction when compared to the face-to-face setting. The study results also suggest developer experience

may be an important factor that should be considered in ensuring the best results when utilizing pure pair programming.

We recognize several limitations in the current study. Most subjects are students in MIS and AIS programs. While the laboratory setting is necessary to create experimental rigor, the subjects' behavior may not be representative of their behavior in a non-contrived work environment. Also, the collaborative software utilized during the experimental tasks may have impacted virtual performance outcomes. The "Create—Design" experimental tasks required subjects to write pseudocode and not actual code. Had subjects written code in a target language, other impacts from the use of pair programming may have been found. This scope also prevents any understanding of performance gains from pair programming in the integration and testing phases of development, which is a stated benefit of the practice. In addition, two, relatively short experimental tasks that are suited to the laboratory setting do not capture the scope and complexity of real world software development, and thus further research is needed in the field in order to generalize these findings to industrial settings.

In future research we will explore the impact of collaborative developer pairing more thoroughly with an eye towards enhancing the performance and satisfaction of the pairs. We plan to explore the impacts of other individual differences such as developer conflict style on performance outcomes. An initial study on the impact of pair programming conflicts is reported in [8]. In addition, we will further investigate the barriers and impediments to virtual pair programming based on the low performance and satisfaction found in the current study. One possible source of problems with the virtual setting is the overhead from use of the collaboration software.

Future research should also investigate other issues with pair programming. Experiments that require subjects to write actual code in a target language would help build a fuller understanding of pair programming. Since pair programming is viewed by some as a way to train new or less experienced developers by pairing them with more experienced developers, future research should explore the impact on performance of programming pairs of all types (high-high, low-low, and high-low). Since there was variation in faithfulness to the method despite equal training for all subjects on pair programming, the source of this variation in faithfulness is also a potential focus for new studies.

## Appendix I: Experimental tasks

### Experimental Task I—Discount Invoice Module

This module is part of the invoice processing program for a retailer. In this module, any discounts for the customer are computed. At this retailer, there are two ways to earn a discount:

- (1) *total-purchase-amount-discount*—if the total amount of purchases, pre-tax, is greater than an established amount; and
- (2) *product-specific-discount*—if the total number of purchases of designated products is greater than a second, set number of purchases. This second type of discount is given to encourage sales of certain products.

If the customer's purchases earn both types of discounts, the second, product-specific discount is computed first. If the second discount reduces the sale to below the set amount for the first discount, the customer does not get the first, total purchase amount discount.

These values are passed to this module:

- Total1 = pre-tax total of the prices of one or more items purchased on the invoice
- Total\_num = total number of purchases of designated products
- Total2 = total purchase price of purchases of designated products
- Discount\_level1 = established amount of total purchases required to earn a discount
- Discount\_level2 = established number of purchases of designated products required to earn a discount
- Discount1 = percentage discount for total-purchase-amount-discounts
- Discount2 = percentage discount for product-specific-discounts

You can assume that the values passed to this module do not include negative numbers or zeros, since the input is checked in the other module.

This module should return these values:

- Total\_discount1 = amount of total-purchase-amount-discount

- New\_total1 = new total purchase amount (that reflects the discount(s))
  - Total\_discount2 = amount of product-specific-amount discounts
- a. Prepare a test data set with at least 10 cases for this module.
  - b. Write the pseudocode for this module and check it for accuracy.

### Experimental Task II—Sales Report Module

This module produces a sales report for a car dealership. For each car sold, the following information is stored in the SALES file, which is sorted by date and time of a car sale.

| Variable name | Description                                                                          | Type and format              |
|---------------|--------------------------------------------------------------------------------------|------------------------------|
| Auto_ID       | Automobile ID number                                                                 | 15 alphanumeric              |
| Sale_Date     | Date of Sale                                                                         | date field                   |
| Sale_Time     | Time of Sale                                                                         | HH:MM                        |
| Make          | Make of Car                                                                          | 10 alphanumeric              |
| Model         | Model of Car                                                                         | 10 alphanumeric              |
| Veh_Type      | Vehicle Type                                                                         | 5 alphanumeric               |
| S_Name        | Salesperson Name<br>(formatted: last name, space, first name, space, middle initial) | 20 character                 |
| S_Comm        | Salesperson Commission                                                               | (2) numeric                  |
| S_Price       | Sales price                                                                          | 6 (2) numeric, dollar format |
| Del_Date      | Delivery date                                                                        | date field                   |

The sales manager wants a report on the first day of each month that lists total sales and commissions for each salesperson. There should be just one line for each salesperson, listing name, average commission, total sales for the previous month, and total commission amount. At the end of this list there should be a grand total of all sales and all commissions earned.

- a. For this module, the test data should be a SALES file with many records. Describe the test records you would put into the SALES file in terms of the number of records and their content.
- b. Write the pseudocode for this module and check it for accuracy.

## Appendix II: Scoring templates

### Task I

#### \_\_\_\_\_ a. Test case completion score (0-10)

- 0 = no test cases completed
- 1 point for each test case that meets one of the six conditions
- 4 = points for more complete test cases (i.e., that don't test one of the conditions)
- 5 = all ten test cases and 1 condition tested
- 6 = more conditions tested
- 9 = almost sufficient conditions
- 10 = sufficient conditions tested

**Look at the test data set created for this task, and answer the following questions about the conditions that should be tested:**

1. Is there at least one test case in which both discounts apply?  
If yes, then place one in the column. \_\_\_\_\_
  2. Is there at least one test case in which neither discounts apply?  
If yes, then place one in the column. \_\_\_\_\_
  3. Is there at least one test case in which only the total purchase discount applies?  
If yes, then place one in the column. \_\_\_\_\_
  4. Is there at least one test case in which only the product specific discount applies?  
If yes, then place one in the column. \_\_\_\_\_
  5. Is there at least one test case for the boundary condition for applying the total purchase discount? If yes, then place one in the column. \_\_\_\_\_
  6. Is there at least one test case for the boundary condition for applying the product specific discount? If yes, then place one in the column \_\_\_\_\_
  7. Then, 1 point for each additional test case (up to 4) \_\_\_\_\_
- Total correct cases (0-10) \_\_\_\_\_

Note: If the case is incomplete, (i.e., not all 7 inputs & 3 output variables included in case) then .5 for each case.

#### \_\_\_\_\_ b. Total pseudocode performance (0-10). Rate the pseudocode on the following scale:

- 0 = no pseudocode written
- 3 = some calculations correct, but not all + incorrect sequencing
- 5 = discounts calculated correctly, but incorrect sequencing of calculations
- 6 = better sequencing
- 7 = all calculations correct, some sequencing correct, but not all
- 9 = almost complete code, includes most logical components
- 10 = complete code (includes all 9 logic components specified)

**Check the pseudocode for accuracy. The logic should include these key parts, in this sequence:**

Set Total\_discount1 and Total\_discount2 to 0

Complete the product-specific discount first

    Check to see if Total\_num is greater than Discount\_level2

        If it is, then Total2\*Discount2 to calculate Total\_discount2

        Subtract Total\_discount2 from Total1 to calculate New\_total1

    Else New\_total1 = Total1

    End if

Compute the total-purchase-amount discount second

    Check to see if New\_total1 is greater than Discount\_level1

        If it is, then NewTotal1\*Discount1 to calculate Total\_discount1

        Subtract Total\_discount1 from New\_total1 to calculate New\_Total1

    End if

END

**Sequencing required (Must be in Order)**

\_\_\_\_\_ Set Total\_discount1 and Total\_discount2 to 0 at beginning

\_\_\_\_\_ Compute total-purchase-amount discount before product-specific discount

\_\_\_\_\_ Subtract Total\_discount2 from Total1 prior to computing total-purchase amount discount

Calculations required

\_\_\_\_\_ Check for Total\_num>Discount\_level2

\_\_\_\_\_ Calculation of Total\_discount2

\_\_\_\_\_ Check for New\_total1 (or whatever variable name they used to represent total purchase amount minus any product-specific discount) > Discount\_level1

\_\_\_\_\_ Calculation Total\_discount1

\_\_\_\_\_ Subtract Total\_discount2 from New\_total1 (or whatever variable name they used to represent total purchase amount minus any product-specific discount) to get New\_total1 (the variable that is returned)

\_\_\_\_\_ Calculate New\_total1 if no discounts are applied

\_\_\_\_\_ Total Correct

**Task II**

\_\_\_\_\_ **a. Test case completion score (0-10)**

0 = no test cases described

1 point for describing a need for test cases for one of the five conditions specified

3 = all ten test cases complete, but no calculations tested

5 = all ten test cases complete and some conditions tested

7 = more test conditions

9 = nearly sufficient conditions tested

10 = completely sufficient test conditions met

Note: If a test case is not complete (all 10 record elements), then .5 for each test case

**Look at the test data set created for this task, and answer the following questions:**Correct Column

Do they say there should be 10 or more test cases?

If yes, then place a one in the column. \_\_\_\_\_

\* Is the test record content complete or shown (then add 1 point)? For each piece of content that is missing, assess one error (subtract 1 point) and place it in the error column.

Automobile ID number \_\_\_\_\_

Date of Sale \_\_\_\_\_

Time of Sale \_\_\_\_\_

Make of Car \_\_\_\_\_

Model of Car \_\_\_\_\_

Vehicle Type \_\_\_\_\_

Salesperson Name \_\_\_\_\_

Salesperson Commission \_\_\_\_\_

Sales price \_\_\_\_\_

Delivery date \_\_\_\_\_

Conditions

1. Did they note that at least two records need to have the salesperson name? \_\_\_\_\_
2. Did they note that at least one record needs to have a sales date within the last month? \_\_\_\_\_
3. Did they note that at least one record needs to have a sales date not within the last month? \_\_\_\_\_
4. Did they include a test record in which one of the required content areas (i.e., fields or variables) in the record is empty or = zero? \_\_\_\_\_  
Assess one error for each field that is not included (0-10)
5. Did they include a test record in which one of the required content areas (i.e., fields or variables) in the record is of the wrong data type? \_\_\_\_\_  
Assess one error for each field that is not checked (0-10)
6. Give 5 points for each additional case described (up to 5) \_\_\_\_\_

Total correct cases \_\_\_\_\_

\* If the test case described is incomplete (i.e., not all these elements) then .5 for each case

Categorize required components

- |                |       |                                                                                                                                                   |
|----------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| File handling  | _____ | Sort the SALES file by salesperson                                                                                                                |
| Calculation    | _____ | Initialize grand totals to zero at the beginning                                                                                                  |
| Sequencing     | _____ | Use some kind of iteration structure (loop, e.g., DO WHILE-END DO) to read through the working file (that is sorted SALES file)- or dbms approach |
| File handlings | _____ | To deal with the first record read file from                                                                                                      |
| Logic          | _____ | Check for salesperson name change                                                                                                                 |
| Calculation    | _____ | After salesperson name change, calculate average commission                                                                                       |
| Printing       | _____ | After salesperson name change, print our report line                                                                                              |

|             |       |                                                                                              |
|-------------|-------|----------------------------------------------------------------------------------------------|
| Calculation | _____ | After salesperson name change, reinitialize working Variables                                |
| Logic       | _____ | Failure to check to see if each record to be included in the report is within the last month |
| Calculation | _____ | For records that are within the last month, calculate number of cars sold                    |
| Calculation | _____ | For the records that are within the last month, calculate monthly commission                 |
| Calculation | _____ | For the records that are within the last month, calculate monthly sales                      |
| Calculation | _____ | For records that are within the last month, calculate the grand total commission             |
| Calculation | _____ | For the records that are within the last month, calculate grand total sales                  |
| Printing    | _____ | Deal with possible need to print last report line after last record is read                  |
| Printing    | _____ | Print out grand totals at end                                                                |
|             | _____ | Total Correct                                                                                |

\_\_\_\_\_ **b. Total pseudocode performance (0-10) Rate the pseudocode on the following scale:**

- 0 = no pseudocode written
- 3 = some calculations correct, but not all + incorrect file handling, sequencing, logic, printing
- 5 = calculations correct, but incorrect file handling, sequencing, logic, printing
- 6 = correct file handling (open and close file)
- 7 = all calculations correct, some file handling, sequencing, logic, printing, but not all
- 8 = better file handling
- 9 = almost complete code, approximately 14 components
- 10 = complete code (includes all 16 required components specified)

Note: File handling may be specified as sequential reads from a flat file, OR reads from a data base management system

**Check the pseudocode for accuracy. The logic should include these key parts, in this sequence:**

```

Sequential read from flat file approach
Sort the SALES file by S_Name (salesperson name) and put the result in some working
file
Set record number to zero
Set working variables for grand total sales and grand total commissions to zero
DO WHILE records exist in the working file (the sorted SALES file)
    Increment record number
    READ record (brings in all 10 variables)
    If record number does not equal 1 AND
        S_Name is not equal to working variable for current salesperson name
        Calculate average commission by dividing monthly-commission
        By no cars sold
  
```

```

        Print working variables for current salesperson name, average
        Commission, monthly sales, and monthly-commission
        Set working variables (monthly-commission, monthly sales, no
        cars-sold) to zero
    End if
    If Sale_Date is within last month
        Set S_Name to working variable for current salesperson name
        Increment no-cars-sold by 1
        Calculate commission =S_Comm * S-Price
        Add commission to monthly-commission
        Add S_Price to monthly-sales
        Add commission to grand total commission
        Add S_Price to grand total sales
    End if
END DO

If no-cars-sold does not equal zero
    Print working variables for current salesperson name, average commission,
    monthly-sales, and monthly-commission
End If

Print grand total sales and grand total commission

```

Also possible- file handling based on reads from a data base management system

## References

1. J. Aiken, Technical and human perspectives on pair programming, *ACM SIGSOFT Software Engineering Notes*, 2003, pp. 244–254.
2. M. Aydin, F. Harmsen, K. van Slooten, R. Stegwee, On the adaptation of an agile information systems development method, *Journal of Database Management* 16(4) (2005) 24–40. October/December.
3. K. Beck, *Extreme Programming Explained*. (Addison-Wesley, Boston, 2000).
4. B. Boehm and R. Turner, Using risk to balance agile and plan-driven methods, *IEEE Computer*, June 2003, pp. 57–66.
5. B. Boehm, R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*. (Addison-Wesley, Boston, 2004).
6. M. Biggs, Pair programming: development times two, *InfoWorld*, Framingham, July 24, 2000.
7. F. Brooks, *The Mythical Man-Month*, 2nd Edition. (Addison-Wesley, Reading, MA, 1995).
8. M. Domino, R. Collins, A. Hevner and C. Cohen, Conflict in collaborative software development, in: *Proceedings of the SIGCPR/SIGMIS 2003 Conference*, Philadelphia, PA, April 2003.
9. J. Erickson, K. Lyytinen and K. Seng, Agile modeling, agile software development, and extreme programming: the state of research, *Journal of Database Management*, 16(4), October/December 2005, pp. 88–100.
10. B. Fitzgerald, N. Russo and T. O’Kane, An empirical study of system development method tailoring in practice, in: *Proceedings of the Eighth International Conference on Information Systems*, December 2000, pp. 187–194.
11. B. Fitzgerald, N. Russo and T. O’Kane, Software development tailoring at Motorola, *Communications of the ACM* 46(4) (2003) 65–70 April.
12. N. Flor and E. Hutchins, Analyzing distributed cognition in software teams: a case study of team programming during perfective software maintenance, in: *Proceedings of the Fourth Annual Workshop on Empirical Studies of Programmers*, J. Koenemann-Belleveau et al. (eds.) Ablex Publishing, Norwood, NJ, 1991, pp. 36–59.
13. A. Gopal, R. Bostrom and W. Chin, Applying adaptive structuration theory to investigate the process of group support systems use, *Journal of Management Information Systems*, 9(2), Winter 1992–1993, pp. 45–62.
14. G. Hedin, L. Bendix and B. Magnusson, Teaching extreme programming to large groups of students, *Journal of Systems and Software* 74 (2005) 133–146.
15. S. JEX, *Organizational Psychology: A Scientist-Practitioner Approach*. (John Wiley and Sons, Hoboken, NJ, 2002) pp. 65–90.
16. L. Lindstrom and R. Jeffries, Extreme programming and agile software development methodologies, *Information Systems Management* 21 (2004) 41–52.
17. J. Manzo, The Odyssey and other code science success stories, *CrossTalk—The Journal of Defense Software Engineering* 15(10) (2002) 22–24 October.
18. P. McBreen, *Questioning Extreme Programming*. (Addison-Wesley, Reading, MA, 2003).
19. J. McGrath, *Groups: Interaction and Performance*. Prentice-Hall, Englewood Cliffs, NJ, 1984).
20. M. Muller and F. Padberg, On the economic valuation of XP Projects, in: *Proceedings of the Ninth European Software Engineering Conference*, September 2003, pp. 168–177.
21. J. Nawrocki and A. Wojciechowski, Experimental evaluation of pair programming, in: *Proceedings of the 12th European Software Control and Metrics Conference*, ESCOM 2001, London, April 2001.

22. J. Nosek, The case for collaborative programming, *Communications of the ACM* 41(3) (1998) 105–108 March.
23. A. Parrish, R. Smith, D. Hale and J. Hale, A field study of developer pairs: productivity impacts and implications, *IEEE Software*, September/October 2004, pp. 76–79.
24. M. Poole and G. DeSanctis, Use of group decision support systems as an appropriation process, in: *Proceedings of the Twenty-Second Annual Hawaii International Conference on System Sciences*, January 1989.
25. M. Poole and G. Desanctis, Understanding the use of group decision support systems: the theory of adaptive structuration. In: C. Steinfeld and J. Fulk (eds.) *Organizations and Communication Technology*, (Sage, Newbury Park, CA, 1990) pp. 173–193.
26. C. Rolland and N. Prakash, A proposal for context-specific method engineering. In: S. Brinkkemper, K. Lyytinen and R. Welke (eds.) *Method Engineering: Principles of Method Construction and Tool Support*, (Chapman and Hall, Atlanta, 1996).
27. K. Siau, Information modeling and method engineering: a psychological perspective, *Journal of Database Management*, 10(4) (1999) 44–50.
28. D.I.K. Sjøberg, J.E. Hannay, O. Hansen, V.B. Kampenes, A. Karahasanovic, N-K. Liborg and A.C. Rekdal, A survey of controlled experiments in software engineering, *IEEE Transactions on Software Engineering* 31(9) (2005) 733–753 September.
29. D. Turk, R. France and B. Rumpe, Limitations of agile software processes, in: *Proceedings of the Third International Conference on eXtrememe Programming and Agile Processes in Software Engineering*, Sardinia, Italy, 2002, pp. 43–46.
30. D. Turk, R. France and B. Rumpe, Assumptions underlying agile software-development processes, *Journal of Database Management* 16(4) (2005) 62–87 October/December.
31. A. Venkatesh and N. Vitalari, An emerging distributed work arrangement: an investigation of computer-based supplemental work at home, *Management Science* 38(12) (1992) 1687–1706 December.
32. L. Williams, R. Kessler, W. Cunningham and R. Jeffries, Strengthening the case for pair programming, *IEEE Software*, July/August 2000, pp. 19–25.